Start here | bankers.c

```c
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int allocation[n][m], maximum[n][m], need[n][m];
    int available[m], request[m];
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }
    printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &maximum[i][j]);
        }
    }
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++)
    {
        scanf("%d", &available[i]);
    }
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
    int process;
    printf("\nEnter process number making request: ");
```

Start here | bankers.c

```
46          printf("\n");
47      }
48      int process;
49      printf("\nEnter process number making request: ");
50      scanf("%d", &process);
51      printf("Enter Request Vector:\n");
52      for(i = 0; i < m; i++)
53      {
54          scanf("%d", &request[i]);
55      }
56      for(i = 0; i < m; i++)
57      {
58          if(request[i] > need[process][i])
59          {
60              printf("\nError: Process exceeded maximum claim.\n");
61              return 0;
62          }
63      }
64      for(i = 0; i < m; i++)
65      {
66          if(request[i] > available[i])
67          {
68              printf("\nResources not available. Process must wait.\n");
69              return 0;
70          }
71      }
72      for(i = 0; i < m; i++)
73      {
74          available[i] -= request[i];
75          allocation[process][i] += request[i];
76          need[process][i] -= request[i];
77      }
78      int work[m], finish[n], safeSeq[n];
79      for(i = 0; i < m; i++)
80      {
81          work[i] = available[i];
82      }
83      for(i = 0; i < n; i++)
84      {
85          finish[i] = 0;
86      }
87      int count = 0;
88      while(count < n)
89      {
90          int found = 0;
91          for(i = 0; i < n; i++)
92          {
93              if(finish[i] == 0)
```

Start here | bankers.c

```
91          for(i = 0; i < n; i++)
92          {
93              if(finish[i] == 0)
94              {
95                  int possible = 1;
96
97                  for(j = 0; j < m; j++)
98                  {
99                      if(need[i][j] > work[j])
100                     {
101                         possible = 0;
102                         break;
103                     }
104                 }
105                 if(possible)
106                 {
107                     for(k = 0; k < m; k++)
108                     {
109                         work[k] += allocation[i][k];
110                     }
111                     safeSeq[count] = i;
112                     count++;
113                     finish[i] = 1;
114                     found = 1;
115                 }
116             }
117         }
118         if(found == 0)
119         {
120             break;
121         }
122     }
123     if(count == n)
124     {
125         printf("\nSystem is in SAFE state.\n");
126         printf("Safe Sequence: ");
127         for(i = 0; i < n; i++)
128         {
129             printf("P%d", safeSeq[i]);
130             if(i != n - 1)
131             {
132                 printf(" -> ");
133             }
134         }
135         printf("\nRequest can be granted immediately.\n");
136     }
137     else
138     {
139         printf("\nSystem is NOT in safe state.\n");
```

Start here | *bankers.c

```
                {
                    if(need[i][j] > work[j])
                    {
                        possible = 0;
                        break;
                    }
                }
                if(possible)
                {
                    for(k = 0; k < m; k++)
                    {
                        work[k] += allocation[i][k];
                    }
                    safeSeq[count] = i;
                    count++;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(found == 0)
        {
            break;
        }
    }
    if(count == n)
    {
        printf("\nSystem is in SAFE state.\n");
        printf("Safe Sequence: ");
        for(i = 0; i < n; i++)
        {
            printf("P%d", safeSeq[i]);
            if(i != n - 1)
            {
                printf(" -> ");
            }
        }
        printf("\nRequest can be granted immediately.\n");
    }
    else
    {
        printf("\nSystem is NOT in safe state.\n");
        printf("Request cannot be granted.\n");
    }
    return 0;
}
S
```

```
"C:\Users\BMSCECSE-L3-15\D
Enter number of processes: 5
Enter number of resource types: 4

Enter Allocation Matrix:
2
0
0
1
3
1
2
1
2
1
0
3
1
3
1
2
1
4
3
2

Enter Maximum Matrix:
4
2
1
2
5
2
5
2
2
3
1
6
1
4
2
4
3
6
6
5

Enter Available Resources:
3
3
2
```

```
"C:\Users\BMSCECSE-L3-15\D
4
2
4
3
6
6
5

Enter Available Resources:
3
3
2
1

Need Matrix:
2 2 1 1
2 1 3 1
0 2 1 3
0 1 1 2
2 2 3 3

Enter process number making request: 1
Enter Request Vector:
1
1

0
0

System is in SAFE state.
Safe Sequence: P0 -> P3 -> P4 -> P1 -> P2
Request can be granted immediately.

Process returned 0 (0x0)   execution time : 134.619 s
Press any key to continue.
```